Mini-projet Perceptron - Unix

Frédéric Rousseau

2026 – 2027

Mini-projet Perceptron

Sujet

- Développer plusieurs versions d'une application en utilisant les IPC Unix de communication et synchronisation
  - A but pédagogique uniquement, la décomposition en processus n'étant pas très efficace pour ce projet

Prérequis

- Algorithmique et Programmation (S5, S6)
- Programmation système sous Unix (CM, TD)

Les séances

- 3 x 2h les mardis matin 13 et 20 oct., 2 nov., salle 216

Au moins autant de travail personnel

- Projet à réaliser en individuel, ou par binôme
- Encadrement (et évaluation) : Frédéric Rousseau

Evaluation Unix IESE4

Examen écrit : 60%

- En fin de module, examen écrit 2h, 1 feuille A4 recto-verso *manuscrite*

Mini-projet : 40%

- Rendu en fin de chaque séance
- L'évaluation sera faite principalement sur le rendu en séance 3
- Pas de rapport, pas de soutenance
- Evaluation sur les objectifs (ou le dépassement !) de programmation et d'exécution
- Evaluation individuelle possible pour un binôme

Objectifs du projet

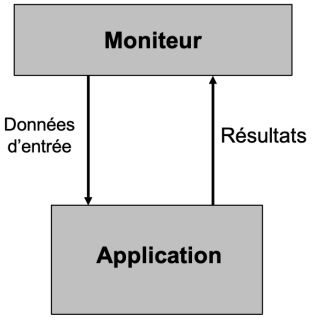
Mise en application des fonctionnalités Unix suivantes :

- Application multi-processus : fork()
- Synchronisation multiprocessus : sémaphores, signaux
- Communication multiprocessus : pipe, mémoire partagée
- Programmation multi fichiers : Makefile

Principe du projet :

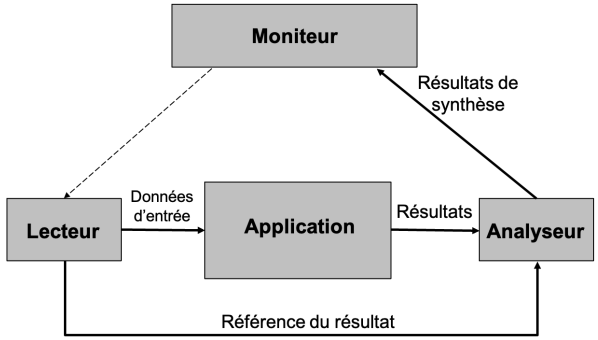
- Vous devez écrire le programme qui respecte un cahier des charges précis
- Vous devez à **chaque séance rendre une version de votre travail**
  - Dépot sur Chamilo ou serveur de la salle 216

Objectif séance 1 - IPC mis en oeuvre



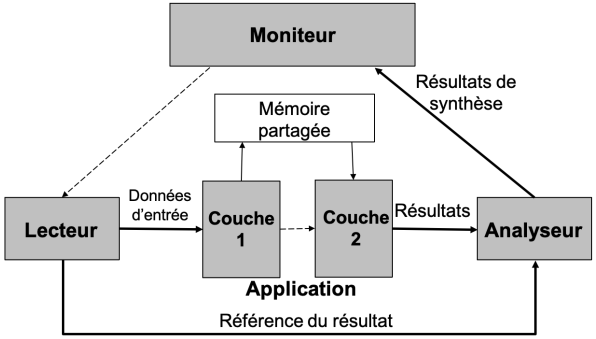
- 2 processus
- Communications par pipe

Objectif séance 2 - IPC mis en oeuvre



- 4 processus
- Communications par pipe
- Synchronisation avec un signal

Objectif séance 3 - IPC mis en oeuvre



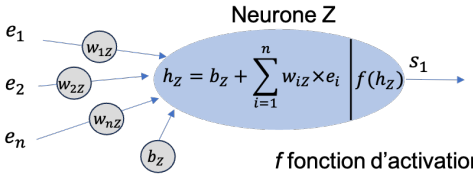
- 5 processus
- Communications par pipe
- Echanges de données entre couche 1 et couche 2 via une mémoire partagée
- Synchronisation avec un signal et par sémaphore entre couche 1 et couche 2

Description générale du perceptron

- Un perceptron est un modèle mathématique d'un neurone biologique
- Par abus de langage, il représente aussi un réseau de neurones, généralement utilisé pour des problèmes de classification
- Il a été inventé en 1957 par Franck Rosenblatt (labo d'aéronautique de l'univ. de Cornell)
- Si un problème est linéairement séparable, un théorème assure que la règle du perceptron permet de trouver une séparation entre les classes
- La méthode d'apprentissage du perceptron est classiquement l'algorithme du gradient stochastique (rétropropagation du gradient)

Perceptron monocouche

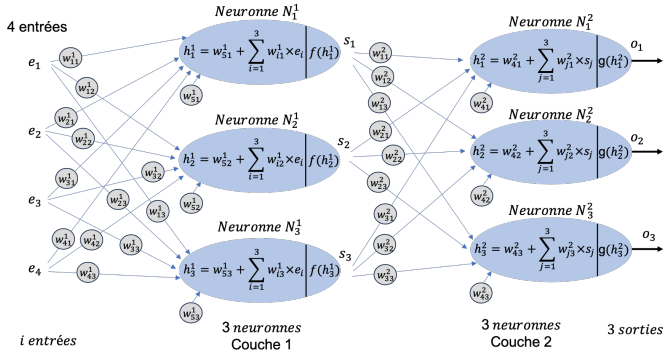
- La sortie d'un perceptron est la somme pondérée des entrées, à laquelle on applique un biais. On applique à ce résultat partiel une fonction d'activation.



- La fonction d'activation correspond à un seuil de stimulation, qui une fois atteint entraîne la réponse du neurone
  - Il s'agit généralement d'une fonction non linéaire (sigmoide, arc tan, ...)

Perceptron multicouche MLP

- Pour de meilleurs résultats, on utilise plusieurs couches
- Exemple avec 4 entrées, 2 couches (3 neurones couche 1, 3 neurones couche 2), 3 sorties



Description générale de l'application

Application à la reconnaissance de fleurs d'Iris

- Il s'agit de reconnaitre à partir de caractéristiques de fleurs d'Iris, l'une des 3 espèces : Setosa, Versicolor, Virginica
- Les caractéristiques des fleurs (données d'entrée) sont : longueur et largeur du sépale, longueur et largeur du pétal
- Un jeu de 150 données (50 de chacune des espèces) sert de base d'apprentissage et de test
- C'est un problème ultra-classique d'utilisation du perceptron

Application aux problèmes d'Iris

Quelques contraintes sur le réseau

- 2 couches
  - 3 neurones couche d'entrée, 3 neurones couche de sortie
- Fonction d'activation *softmax* pour la couche de sortie
- Fonction d'activation *tanh* pour la couche d'entrée
- Attention à normaliser les valeurs d'entrée :
  - v_norm = (valeur _d'entrée - moyenne) x écart_type

Ce qui vous est donné

- Fichier .csv contenant les caractéristiques des fleurs d'iris
- Au moins 1 jeu de coefficients avec le taux de reconnaissance (la partie apprentissage a déjà été faite)
- Le perceptron aura 3 sorties, chacune correspondante à une famille de fleur

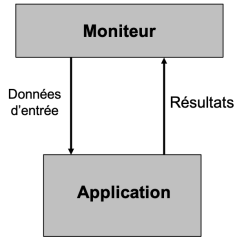
Préparation

Avant la première séance :

- 1 : Développement en C du programme du perceptron 2 couches (4 entrées, 3 neurones couche 1, 3 neurones couche 2, 3 sorties).
- 2 : Validation du modèle avec les coefficients donnés et les deux jeu de tests (valeurs normalisées) fournis
- 3 : (pour la séance 2) : Ecrire une fonction permettant de récupérer les données contenues dans le fichier data_iris.csv

Attendus séance 1

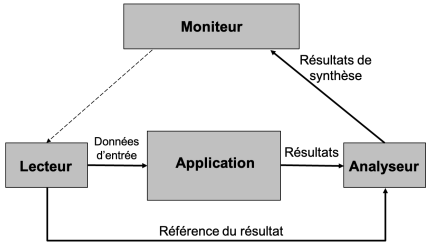
- Mise en oeuvre dans un processus *Application* de votre programme C
- Les données sont gérées par le processus *Moniteur* (à écrire)
- Les données sont envoyées via 4 *pipes* vers le processus *Application*
- A la fin de chaque itération de calcul, le processus *Application* envoie les 3 sorties via 3 *pipes* au *Moniteur*
- En comparant les résultats de chaque calcul avec la valeur attendue, le processus *Moniteur* donne la statistique de classification correcte et affiche ce résultat
- Vous pouvez extraire du fichier *data_iris.csv* toutes les valeurs pour utiliser un ou plusieurs tableaux. Les valeurs d'entrées sont brutes, donc à normaliser.



Attendus séance 2

Le problème est maintenant décomposé en 4 processus

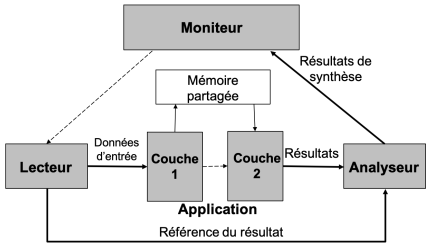
- A la réception de SIGUSR1, un processus *Lecteur* envoie dans les 4 pipes les valeurs d'entrée. Ce même processus *Lecteur* envoie dans 1 pipe la sortie attendue à un processus *Analyseur*
- Le processus *Lecteur* pourra récupérer les données directement dans le fichier CSV original (à normaliser !)
- A chaque itération, le processus *Analyseur* récupère les 3 sorties du processus *Application* et compare avec la référence de résultat
- A la fin du parcours de toutes les données, le *Moniteur* récupère la valeur statistique de classification et l'affiche à l'écran



Attendus séance 3

Le problème est maintenant décomposé en 5 processus

- Le processus *Application* est décomposé en 2 processus : couche 1 et couche 2
- Les résultats produits par la couche 1 sont placés dans une mémoire partagée
- Quand tous les résultats sont écrits, le processus couche 1, via un sémaphore indique au processus couche 2 qu'il peut faire le traitement
- Le processus couche 2 récupère ses données d'entrée dans la mémoire partagée
- La mémoire partagée peut avoir une structure de file circulaire



Décomposition du développement

- Pour préparer
  - Ecrire un programme C du réseau proposé (4 entrées, 3 sorties, 2 couches (3 et 3 neurones)
  - Valider votre modèle avec le jeu de test fourni
- Attendus fin de séance 1
  - 1 processus *Moniteur*, 1 processus *Application*
  - Mise en oeuvre : 2 processus et des *pipes*
- Attendus fin de séance 2
  - 1 processus *Lecteur*, à la réception de SIGUSR1, démarre la classification
  - Le processus *Moniteur* affiche le résultat final
  - Mise en oeuvre : 4 processus, 1 signal, des *pipes*
- Attendus fin de séance 3 (version 2025)
  - Décomposition du processus *Application* en 2 processus distincts correspondant aux 2 couches du perceptron
  - Les échanges entre le processus couche 1 et le processus couche 2 sont faits à travers une *mémoire partagée*, et sont synchronisés par 1 *sémaphore*
  - Mise en oeuvre : 5 processus, 1 *signal*, des *pipes*, 1 *sémaphore*, 1 *mémoire partagée*

Les questions à se poser

- Comment, par qui et quand sont créés les différents éléments de communication et de synchronisation des processus ?
- Comment terminer proprement vos programmes en fonctionnement normal ?

Pour aller plus loin

- Comment terminer proprement vos programmes si un CTRL C est émis ?
- Peut-on rendre générique le perceptron pour placer les coefficients des 2 couches dans la mémoire partagée ?
- Peut-on améliorer les programmes ou rendre plus générique le modèle (nombre de couches, nombre de neurones par couche, ...) ?

Mini-projet Perceptron - Unix IESE4

Iris setosa

Iris versicolor

Iris virginica